Concepts-Adjacent Problems

Document #:	P1900R0
Date:	2019-10-06
Project:	Programming Language C++ EWG
Reply-to:	Barry Revzin <barry.revzin@gmail.com>

1 Introduction

This paper is not a proposal. It does not offer any concrete suggestions of additions, changes, or removals from C++20. I don't yet have specific solutions to offer. My goal with this paper is to present a set of problems that I believe should be solved, in an effort to both raise awareness about them and to motivate finding a solution for them.

One of the marquee features of C++20 is Concepts. C++20 Concepts are a language feature that was set out to solve the problem of constraining templates and overload sets. They offer many significant improvements over the C++17 status quo; there are many problems with become straightforwardly and easily solveable with constraints that used to be either impossible or of sufficient difficulty and verbosity that only a small handful of experts, that were sufficiently motivated, could solve them:

— Constraining non-template functions of class templates. Notably, constraining the special member functions of class templates.

— Having multiple constrained overloads, without having to be exceedingly careful in ensuring that all of them are disjointly constrained.

— Constraining class template partial specializations, both in the multiply constrained case (as above) and without having to have the primary class template opt-in to such constrained specializations (i.e. as `std::hash<T>` does not).

Those are big wins. Those are *really* big wins.

And even in the cases where C++17 could solve the problem just fine, writing constraints with `concept`s and `requires` is just a lot nicer looking that writing constraints with `std::enable_if`.

But there are many problems in the world of generic programming that are not simply constraining overload sets - and C++20 Concepts does not solve them. But they're such closely related problems, that perhaps a future incarnation of them should. I'm going to go through a few such problems: associated types, explicit opt-in/opt-out, customization, and type erasure.

I also want to make something clear up front. Many of the examples in this paper come from Ranges, so it might be easy to conclude from this that I consider Ranges to be some overly complex mess. This conclusion is the furthest thing from the truth. I use Ranges as examples precisely because I consider Ranges to be the most complete, well thought out, and generally best possible use of concepts there is. This paper exists because I do not know if it is possible to do better with the language tools that we have.

2 Associated Types

A C++20 `concept` is a predicate on types (or values or templates, but let's just say types for simplicity). It simply provides a yes or no answer to a question. For many concepts, that is completely sufficient. Some type `T` either models `std::equality_comparable` or it does not - there's no other relevant information in that question. It's easy to come up with many examples like this.

However, for certain concepts, a yes/no really isn't enough. There's more information that you need to have. Take a `concept` like `std::invocable`. We can say that some type `F` models `std::invocable<int>` or not - that tells us if we can call it with `42`, or not. But there's one especially useful piece of information to have in addition to this. It's not just: *can* I call this thing with `42`. There's also: what do I get when I do? What is the resulting type of this invocation? We call this result type an *associated type* of the concept `invocable`.

It's really rare to want to constrain on an invocable type but not care at all about what the result of that invocation is. Typically, we either need to add further constraints on the result type or we need to take the result of the invocation and do something with it.

For instance, the concept `std::predicate` is a refinement of `std::invocable` such that the result type of that invocation models `std::convertible_to<bool>`.[1] How do we check that? We have to use a type trait:

```
template <typename F, typename... Args>
  concept predicate = std::invocable<F, Args...> &&
        std::convertible_to<std::invoke_result_t<F, Args...>, bool>;
```

Now here's the question: what is the relationship between the `concept std::invocable` and the type trait `std::invoke_result_t`? None. There is no relationship. How did I know the correct type trait to use in this situation? I just did. It's just something I had to know.

Some readers might quibble at this point that this isn't a real problem - after all, I introduced this as wanting to know the "result type" of "invocable", so perhaps it's not at all surprising that this thing is spelled `invoke_result_t`. But we typically want to have closely associated entities actually be more closely associated than simply having similar names. These two aren't even in the same header.

Let's take a different example. Another marquee C++20 feature is the introduction of Ranges. While `std::invocable` has one associated type (even if we cannot express that association in the language), the core concept of Ranges - `std::range` - has several:

— the iterator type

— the sentinel type (not necessarily the same type as the iterator type)

— the value type

— the reference type

— the iteration category

— the difference type

Pretty much every function template that takes a `std::range` will need to use at least one of these associated types. How do we get this information today? Again, we have to rely on the use of type traits. And you just have to know what these type traits are:

— `iterator_t`

— `sentinel_t`

— `range_value_t`

— `range_reference_t`

— `range_difference_t`

And I don't know if there's a type trait for the category.

Because we don't have a way to express associated types, we have to solve this problem with a proliferation of type traits. Which means a much larger surface area of things people have to know in order to write any kind of code. Or worse, instead of using the type traits, people resort to reimplementing them - possibly incorrectly.

§ 3 Explicit opt-in/opt-out

C++20 concepts are completely implicit. But sometimes, implicit isn't really what we want. We have `explicit` for type conversions precisely because we understand that sometimes implicit conversions are good and safe and sometimes they are not. Type adherence to a concept is really no different. There are many cases where a type might fit the *syntactic* requirements of a concept but we don't have a way of checking that it meets the *semantic* requirements of a concept, and those semantics might be important enough to merit explicit action by the user.

One way to allow explicit control in concept definitions is to defer to type traits. For instance, the `view`, `sized_range`, and `sized_sentinel_for` concepts in the standard library come with type traits that allow for explicit tuning:

```
template<class T>
  concept view =
    range<T> && semiregular<T> && enable_view<T>;

template<class T>
  concept sized_range =
    range<T> &&
    !disable_sized_range<remove_cvref_t<T>> &&
    requires(T& t) { ranges::size(t); };

template<class S, class I>
  concept sized_sentinel_for =
    sentinel_for<S, I> &&
    !disable_sized_sentinel<remove_cv_t<S>, remove_cv_t<I>> &&
    requires(const I& i, const S& s) {
      { s - i } -> same_as<iter_difference_t<I>>;
      { i - s } -> same_as<iter_difference_t<I>>;
    }
```

Not all `semiregular` `ranges` are `views`, we need an extra knob to control. That's what `enable_view<T>` is for: it's a type trait to help opt types out of being `views`. The specializations that come with the standard library help *exclude* types that provide different deep `const` access (since deep `const`-ness implies ownership, e.g. `std::vector<T>`) and then other specific containers in the standard library that don't provide deep `const` because their only provide `const` access (e.g. `std::set<T>`), but also to *include* types that can opt-in directly (i.e. by way of either inheriting from `view_base` or otherwise specializing `enable_view`).

This ability to explicitly state which `semiregular` `ranges` are and are not `views` seems fundamental, but we still need a type trait for that.

The `sized_range` concept illustrates similar functionality. The semantics of `sized_range` are that `x.size()` is `O(1)`. If a `range` has a `size()` member function that is *not* constant-time, then it should opt out of being a `sized_range`. Pre-C++11 `std::list` is one such example container. But again, we don't have this control through the `concept`, we need to come up with some external mechanism, the easiest of which is a type trait.

There is a another Ranges concept that requires explicit opt-in, but does so without a type trait. And that is *`forwarding-range`*. The semantics of a *`forwarding-range`* are that the iterators' validity is not tied to the lifetime of the `range` object. This is a purely semantic constraint that is impossible to determine merely syntactically, and a semantic that would be dangerous to get wrong at that, so it's precisely that kind of thing that merits an explicit opt-in. The design is: a (non-reference) type `R` satisfies *`forwarding-range`* if there is are *non-member* `begin()` and `end()` functions that can be found by argument-dependent lookup that either take an `R` by value or by rvalue reference. While this mechanism is not exactly a type trait, it does provide the same function: we need an explicit opt-in mechanism for a `concept`.

Because we don't have a way to express explicit opt-in or opt-out, we have to solve this problem with either a proliferation of type traits or more bespoke solutions. Which again means a much larger surface area of things people have to know in order to write any kind of code.

3.1 Opting into customization

This section largely dealt with the problem of being explicit with regards to opting into and out of a concept, as a whole. How do we opt `vector` and `set` out of `view`? How do we opt `subrange` and `string_view` into *`forwarding-range`*?

But there's also a different kind of opting into concepts to consider: how do we actually opt into `std::range`? The next section will talk about the customization problem as a whole, but let's just focus on the opt-in. In order for a type to be a `range`, it needs to have `begin()` and `end()` functions (member or non-member) that return an iterator/sentinel pair. That, in of itself, is well understood:

```
template <typename T>
struct my_vector {
   // ...

   auto begin() -> T*;
   auto end()   -> T*;
```

```
    auto begin() const -> T const*;
    auto end() const   -> T const*;
};
```

But if you think about it, why are those member functions there at all? They are there precisely for the purpose of explicitly opting into what is now the `std::ranges::range` concept. Your type is never *accidentally* a range, it is always on purpose. The first `begin`/`end` pair exists to ensure that `my_vector<T>` is a `std::ranges::range` and the second `begin`/`end` pair exists to ensure that `my_vector<T> const` is a `std::ranges::range`. But there's nothing in the actual code that indicates this relationship at all.

Now you might think that this desire for explicitness is a bit silly at best or verbose at worst. But that's mostly because this example is so well-known. Everybody understands ranges. But what if I added:

```
template <typename T>
struct my_vector { /* ... */ };

template <typename T>
void draw(my_vector<T> const&, std::ostream&, size_t);
```

What is that function doing there? Is it just some non-member function that exists in a vacuum for an application, or is there for some other purpose? Its actual purpose is to satisfy the drawable concept from Sean Parent's "Inheritance is the Base Class of Evil" talk [Parent]. That was the intent of writing it but there's nothing I can write to indicate that the intent of this function is to provide an implementation for that concept (outside of a comment, but nothing with actual semantics), and there's nothing that can check that I did it correctly (was the third argument `size_t` or was it `int`?).

For this specific issue, see also Matt Calabrese's [P1292R0] which does provide a way for a semantic override for a function like this, although it too is separate from the `concept` langauge feature.

§ 4 Customization

Let's got back to `std::ranges::range`. How do we learn how to opt into `std::ranges::range`? The definition of that concept is:

```
template<class T>
  concept range-impl =
    requires(T&& t) {
      ranges::begin(std::forward<T>(t));
      ranges::end(std::forward<T>(t));
    };

template<class T>
  concept range = range-impl<T&>;
```

What does this concept definition tell you about what the interface of `T` has to be in order to satisfy `std::ranges::range`? It doesn't really tell you all that much at all.

The constraints on `T` aren't in the definition of the concept `std::ranges::range`. In order to to determine if `std::ranges::range` is satisfied, you have to go look for what `ranges::begin` and `ranges::end` are and see what their constraints are. How easy is that to do? Let's take a look at the implementation of `begin` in [cmcstl2] (the version for `end` is roughly the same so I'll simply focus the dicussion on `begin`):

```
namespace __begin {
    // Poison pill for std::begin. (See the detailed discussion at
    // https://github.com/ericniebler/stl2/issues/139)
    template<class T> void begin(T&&) = delete;

    template<class T>
    void begin(std::initializer_list<T>) = delete; // See LWG 3258

    template<class R>
    concept has_member = std::is_lvalue_reference_v<R> &&
        requires(R& r) {
            r.begin();
            { __decay_copy(r.begin()) } -> input_or_output_iterator;
        };
```

```
        template<class R>
        concept has_non_member = requires(R&& r) {
            begin(static_cast<R&&>(r));
            { __decay_copy(begin(static_cast<R&&>(r))) } -> input_or_output_iterator;
        };

        template<class>
        inline constexpr bool nothrow = false;
        template<has_member R>
        inline constexpr bool nothrow<R> = noexcept(std::declval<R&>().begin());
        template<class R>
        requires (!has_member<R> && has_non_member<R>)
        inline constexpr bool nothrow<R> = noexcept(begin(std::declval<R>()));

        struct __fn {
            // Handle builtin arrays directly
            template<class R, std::size_t N>
            void operator()(R (&&)[N]) const = delete;

            template<class R, std::size_t N>
            constexpr R* operator()(R (&array)[N]) const noexcept {
                return array;
            }

            // Handle basic_string_view directly to implement P0970 non-intrusively
            template<class CharT, class Traits>
            constexpr auto operator()(
                std::basic_string_view<CharT, Traits> sv) const noexcept {
                return sv.begin();
            }

            template<class R>
                requires has_member<R> || has_non_member<R>
            constexpr auto operator()(R&& r) const noexcept(nothrow<R>) {
                if constexpr (has_member<R>) {
                    return r.begin();
                } else {
                    return begin(static_cast<R&&>(r));
                }
            }
        };
    }

    inline namespace __cpos {
        inline constexpr __begin::__fn begin{};
    }

    template<class R>
    using __begin_t = decltype(begin(std::declval<R>()));
```

Unless you've seen this style of code before and are very familiar with the design, it's probably going to be pretty hard to figure out what you actually need to do. Moreover, for the authors of a concept, this is a lot of fairly complex code! Using C++20 concepts makes this code substantially easier to write and understand than than the C++17 version would have been, but it's still not exactly either easy to write or understand.

The important question is: why does this have to be so complex?

We have two sources of implementation complexity here, in my opinion:

1. In the implementation of `ranges::begin`, we have the explicit opt-in for *`forwarding-range`* mentioned earlier: the poison pill overloads, the specific overload for `basic_string_view` and deleted rvalue array, and the `is_lvalue_reference` constraint on `has_member`.
2. To maximize usability, we want to specify *what* a type must opt into, but not make any restrictions on *how* a type must opt into it.

The first part I already discussed, so let's talk about the second.

A type models `std::range` if it has a `begin()` function that returns an iterator. The question is *how* can a type provide this `begin()`?

We don't want to impose on class authors how their types have to model our concepts - whether member or non-member function - we want it to be up to them. This allows maximal flexibility. But whichever path a type takes, we want to be easy for authors of generic code to write that code without having to go through all this process - either the careful constraints that can be seen in the implementation of `begin` above, or simply adding a free function that calls the member function (as in `std::begin`) and then requiring what Eric Niebler called the "Std Swap Two-Step" [Niebler]:

```
using std::begin;
begin(x);
```

A this point, you might be thinking that the solution that I'm thinking of for this problem is unified function call syntax. But UFCS would not actually solve this problem,[2] we need something else.

As a result, if we want to give people flexibility in how they can opt into concepts (which of course we do), then we cannot even specify our constraints within concepts themselves. We have to defer to function objects with pairs of concepts to handle both member and non-member implementations. None of the code is reusable. We have a fairly simple concept (we're just checking for two functions whose return types have to satisfy other concepts, this isn't, in of itself, especially complex), yet this still takes about 140 lines of code in Casey Carter's implementation.

But customization is a critical piece of generic programming. This seems like something that should be handled by a concepts language feature. The status quo just seems like too much code, that is too complicated and too easy to mess up, to have to write for each and every concept.

Because we don't have a way to directly express customization with concepts, we have to solve this problem with a proliferation of function objects. Which means a much larger surface area of things people have to know in order to write any kind of code. Or worse, instead of using the function objects, people will choose a syntax - leading to under-constrained templates.[3]

§ 5 Type Erasure

The three earlier problems presented are issues with using `concept`s directly - opting in or out, getting more information, being explicit, using them in generic code. An entirely different kind of problem is: how can we use a `concept` to build something else out of it? In the same Sean Parent talk I cited earlier [Parent], he presents an argument against the idea of polymorphic types. To quote some lines from his slides:

> There are no polymorphic types, only a *polymorphic use* of similar types
>
> By using inheritance to capture polymorphic use, we shift the burden of use to the type implementation, tightly coupling components
>
> Inheritance implies variable size, which implies heap allocation
>
> Heap allocation forces a further burden to manage the object lifetime
>
> Indirection, heap allocation, virtualization impacts performance

He then goes on to present a type erasure approach to polymorphic use. This type erasure approach is built on a concept. Not a C++20 language `concept`, but a concept nevertheless. It is even called concept in the slides:

```
struct concept_t {
    virtual ~concept_t() = default;
    virtual concept_t* copy_() const = 0;
    virtual void draw_(ostream&, size_t) const = 0;
};
```

Can we do something like this using C++20 language concepts? The language `concept` might look like this:

```
template <typename T>
concept drawable =
    requires (T const& v, ostream& os, size_t pos) {
```

```
            parent::draw(v, os, pos); // a CPO for member or non-member draw
        };
```

How do we produce a type out of this `concept` which can hold any `drawable` that is also itself `drawable`? Even with full reflection facilities, this seems like an overwhelmingly difficult problem. It's telling that all the reflection-based type erasure work has not been based on `concept`s but has been instead based on simple class definitions with member functions (Louis Dionne presented such an idea at CppCon 2017 [Dionne], Sy Brand implemented this recently using metaclasses [Brand] [Brand.Github], and Andrew Sutton discussed and heavily praised Sy's implementation at CppCon 2019 [Sutton]).

Such a generalized facility, to take one or more `concept`s and synthesized a type erased object out of them so that they do not need to be hand-written, would be incredibly useful. We even have a ready-made case study. C++11 added `std::function`, a type erased, copyable, owning callable. This type has proven very useful. But for C++20, we tried to add two more very similar types:

— A type-erased, move-only, owning callable: `std::any_invocable` [P0288R4]

— A type-erased, non-owning callable: `std::function_ref` [P0792R4]

If we had the ability to take a `concept` and create a type erased type out of it, all of this work would have been trivial. The papers in question would have either just been requests to add alias templates (if we would even need such to begin with):

```
  namespace std {
    template <typename Sig>
    using function = any_concept<invocable<Sig>, sbo_storage<implementation-defined>>;

+   template <typename Sig>
+   using any_invocable = any_concept<invocable<Sig>, move_only_storage>;
+
+   template <typename Sig>
+   using function_ref = any_concept<invocable<Sig>, non_owning_storage>;
  }
```

Concept-driven type erasure is an important use case of `concept`s, one which isn't solved by the language feature we have today. Instead, we have to solve this with a proliferation of types which hand-implement the specific type erasure with the specific storage choice on a case-by-case basis. Because these types are so difficult to write, yet so useful, there is a push to add them to the standard library – and each such type is an independent, slow process.

Potentially, with a future generative metaprogramming language feature, built on [P0707R4] and [P1717R0], we could write a library to avoid hand-implementing type erased objects (as in Sy's example [Brand.Github]). It's just that such a library would not be based on `concept`s, and would either lead to a bifurcation of the contraint system or we would have such a library would inject a `concept`s for us. Either of which seems like an inadequacy of `concept`s.

§ 6 Proposal

As I said in the very beginning of this paper, this paper is not a proposal. I do not have concrete suggestions for how to solve any of these problems (or even vaguely amorphous suggestions). The goal of this paper is instead to present the problems that the concepts language feature could solve, and should solve, but at the moment does not.

But because these problems have to be solved, we end up with proliferations of type traits, bespoke opt-in solutions, customization point objects, and whole classes. The surface area that a programmer needs to know to write good generic code is enormous.

To the extent that that this paper is a proposal, it's a proposal for proposals to solve these problems and a proposal to seriously consider those future proposals. This might mean restarting SG8, or simply taking more EWG time. But they're big problems and solving them could reap huge benefits.

§ 7 References

[Brand] Sy Brand. 2019. I've written a proof-of-concept implementation of Rust-style trait objects in C++ using the experimental metaclasses compiler.
https://twitter.com/tartanllama/status/1159445548417634324?lang=en

[Brand.Github] Sy Brand. 2019. Typeclasses in C++.
https://github.com/tartanllama/typeclasses/

[cmcstl2] Casey Carter. 2019. An implementation of C++ Extensions for Ranges.
https://github.com/CaseyCarter/cmcstl2/blob/43c77f9152c2470f8bc4f820f88ef51639ac2053/include/stl2/detail/range/access.hpp#L41-L100

[Dionne] Louis Dionne. 2013. CppCon 2017: Runtime Polymorphism: Back to the Basics.
https://youtu.be/gVGtNFg4ay0?t=3242

[Niebler] Eric Niebler. 2014. Customization Point Design in C++11 and Beyond.
http://ericniebler.com/2014/10/21/customization-point-design-in-c11-and-beyond/

[P0288R4] Ryan McDougall, Matt Calabrese. 2019. any_invocable.
https://wg21.link/p0288r4

[P0707R4] Herb Sutter. 2019. Metaclasses: Generative C++.
https://wg21.link/p0707r4

[P0792R4] Vittorio Romeo. 2019. function_ref: a non-owning reference to a Callable.
https://wg21.link/p0792r4

[P1292R0] Matt Calabrese. 2018. Customization Point Functions.
https://wg21.link/p1292r0

[P1717R0] Andrew Sutton, Wyatt Childers. 2019. Compile-time Metaprogramming in C++.
https://wg21.link/p1717r0

[Parent] Sean Parent. 2013. GoingNative 2013: Inheritance Is The Base Class of Evil.
https://www.youtube.com/watch?v=2bLkxj6EVoM

[Sutton] Andrew Sutton. 2019. Meta++: Language Support for Advanced Generative Programming.
https://youtu.be/kjQXhuPX-Ac?t=2057

---

1. This is not the current definition of `std::predicate`, but possibly should be, and in any case, the difference isn't relevant for this paper.↩

2. Let's consider the version of UFCS that said that member functions can find non-member functions. What would this mean:

```
int arr[10];
arr.begin();
```

Arrays do not have member functions, so we try to find a `begin(arr)`. But `int[10]` doesn't have any associated namespaces. Such a call would only succeed if there were a `begin` in scope. Where would our array overload for `begin` live? It would have to live in global namespace? But even then, we would have to rely on there not being any other `begin`s between where we are and that global declaration otherwise this won't work. To be safe, we'd have to put the array overload somewhere specific and bring it in scope:

```
namespace std {
    template <typename T, size_t N>
    constexpr T* begin(T (&arr)[N]) noexcept { return arr; }
}

int arr[10];
using std::begin;
arr.begin(); // ok, would call std::begin(arr)
```

But at this point, we're doing the Two-Step (because we have to), so we didn't really gain anything from UFCS. The same problem will come up anytime you want to customize a function for things like pointers, arrays, or just fundamental types.↩

3. What I mean is, instead of using `ranges::begin(x)` in generic code which takes a `std::range`, people will write either `x.begin()` or `begin(x)` – both of which

are incorrect.↩